Multi-Label GitHub Issue Classification

Achintya Kattemalavadi
University of California, Berkeley, School of Information
akattema@berkeley.edu

Abstract

Automatically classifying software issues is essential for improving the triage and resolution process in large-scale software projects. This paper presents a transformer-based approach to specific (beyond just "bug", "enhancement", and/or "question") multi-label classification of GitHub issues using issue titles, descriptions, and comment threads as input. A new dataset was constructed from ten popular open-source repositories, containing 94,070 closed issues labeled across 18 standardized categories. Three model architectures were evaluated: BERT, RoBERTa, and T5. BERT achieved the highest classification performance, with a micro F1-score of 0.8606, while T5 offered competitive results and enabled evaluation with generative metrics such as BLEU and ROUGE. RoBERTa underperformed in comparison, particularly on rare labels. Results show that incorporating comment threads improves classification and that significant label imbalance remains a key challenge. The paper concludes with recommendations for future research, including use of auxiliary data, generative models, and dynamic label prediction as issue discussions evolve.

1 Introduction

Managing a large number of issues in software development can be challenging. Tracking previous issues to identify recurring problems is non-trivial (Anvik et al., 2006). During the lifetime of an issue, its categorization may change as more relevant information surfaces. All this together can lead to inefficiencies in assigning issues to personnel and eventually solving them (Zhang and Lee, 2012).

Prior research has focused on classifying issues by three labels: "bug", "enhancement", or "question" (Siddiq and Santos, 2022; Bharadwaj and Kadam, 2022). However, in practice, these categories are too broad given the complexity of modern software systems. In addition, other approaches often rely solely on the issue title and body, not taking into account comments which can change the status of the issue dynamically. The goal of this project is to create a specific (not limited to "bug", "enhancement", or "question"), multi-label classification system for GitHub Issues by utilizing the titles, description, and attached comments.

The eventual application of this research would be to automate nuanced labeling and categorization of issues and tickets, as well as enable dynamic relabeling of issues given updates in comment threads on an issue.

2 Related Work

Previous work has taken a variety of approaches to classify and assign software issues. Zhang and Lee (2012) created concept profiles by using k-means clustering on bug reports. They identified related concepts through cosine similarity based on the combined title and description of each bug report. Common topic terms were then extracted from these clusters, and new bugs were matched to concepts by computing their similarity to the cluster topics. They also assigned issues to specific developers by constructing a social network of developer interactions, where developers were ranked for assignment using graph-based metrics and historical bug-fixing performance on issues with related concepts.

Arya et al. (2019) conducted a qualitative content analysis of issue threads from open-source projects to identify 16 distinct information types commonly found in discussion comments. Their annotated dataset was used to train models to automatically classify discussion comments based on these information types. Results showed that Random Forest classifiers performed well with just conversational features such as comment length, position, and author role, while Logistic Regression had mixed performance using the comment text alone. Building on this foundation, Paing et al. (2022) developed QuerTCI, a tool that integrates GitHub's search API with Arya et al.'s classification models to auto-classify search results for GitHub issue comments.

In parallel, several studies adopted transformer-based models for direct issue classification. Siddiq and Santos (2022) fine-tuned the base uncased BERT model to classify GitHub issues as bugs, questions, and/or enhancements, leveraging the issue title and body as input. Similarly, Bharadwaj and Kadam (2022) worked with the same base BERT model, but masked

code segments, URLs, numbers, and other specific pieces of information with tokens such as CODE, URL, and NUMBER to focus the model on linguistic content.

Nadeem et al. (2021) used RoBERTa as a base for issue classification, choosing not to pre-process inputs aside from concatenating the issue title and body text, and tokenization. Their system was deployed in a GitHub-integrated application capable of automatically labeling new issues in real time.

This project differs from previous efforts in several key ways. First, we expand the input context by incorporating not just the title and body of an issue, but also the full comment thread. More information or solutions/workarounds for an issue are often uncovered in the comments as people collaborate to solve it, sometimes leading to reclassification or reassignment of the issue. Second, we extend the classification task beyond a few coarse labels like "bug" or "enhancement" to include a much broader and more specific set of issue categories. Unlike Bharadwaj and Kadam (2022), we intentionally retain content such as code blocks, numbers, and user mentions, as they often contain contextual signals that are valuable for fine-grained classification. Finally, in addition to using encoder-based models like BERT and RoBERTa, we also explore text-to-text transformer models (e.g., T5) for multi-label classification, enabling experimentation with both classification and generative modeling paradigms.

3 Methodology

3.1 Dataset

A multi-label classification dataset was constructed using ten popular open-source GitHub repositories with a high volume of labeled, closed issues. The selected repositories were:

- flutter/flutter
- aws/aws-cdk
- camel-ai/camel
- public-apis/public-apis
- microsoft/vscode
- tensorflow/tensorflow
- facebook/react-native
- pytorch/pytorch
- zephyrproject-rtos/zephyr
- gogs/gogs

The MiniLM-L6-v2 sentence transformer was used to group semantically similar labels. Only label groups present in at least three repositories were retained. A reverse index was created to map each original label to a standardized label name corresponding to its group after manual verification of the groupings. The final 18 standardized labels were:

- accessibility
- release
- dependency
- good_first_issue
- webview
- oslinux
- revert
- bug
- documentation
- featurerequest
- feature
- help_wanted
- test
- ui
- api
- enhancement
- regression
- security

For each issue, the title, body, comments, and associated labels were extracted. Labels were mapped to their standardized forms. Issues were excluded if they lacked a body or did not include any mapped standardized labels.

The dataset contained 112,133 issues. The bug label appeared in over 50% of all issues, prompting resampling to reduce class imbalance, reducing the final number of issues to 94,070. The final dataset was split into training (70%), validation (20%), and test (10%) subsets while maintaining similar label distribution across splits.

Label	Original Proportion	Resampled
bug	53.63%	46.15%
featurerequest	22.20%	26.42%
enhancement	4.73%	5.62%
accessibility	3.91%	3.91%
revert	2.85%	2.85%

Table 1: Top five most frequent labels in the dataset.

The distribution of the number of labels per issue in each dataset is shown in Table 2. Issues with a single label make up the vast majority of the datasets, at over 90% for each.

Label Count	Train (%)	Validation (%)	Test (%)
1	91.83	91.83	91.59
2	6.84	7.21	6.93
3	0.65	0.70	0.70
4	0.01	0.02	0.01

Table 2: Distribution of label counts per issue across dataset splits.

3.2 Architectures

Three transformer-based architectures were fine-tuned for multi-label issue classification: BERT, RoBERTa, and T5.

BERT (Bidirectional Encoder Representations from Transformers) was selected for the baseline model. It is an encoder-only transformer pre-trained on masked language modeling and next sentence prediction. This architecture enables full bidirectional attention, allowing it to understand the context across multiple fields such as issue titles, descriptions, and comments. Since the classification task requires identifying

relevant labels based on language understanding rather than generating text, BERT's encoder structure aligns with the task. The `bert-base-uncased` variant was used.

RoBERTa (A Robustly Optimized BERT Pretraining Approach) builds upon BERT by removing the next sentence prediction objective, adopting dynamic masking, and being trained on a significantly larger corpus for more epochs. These improvements result in better contextual understanding and robustness across tasks.

T5 (Text-to-Text Transfer Transformer) was introduced as a fundamentally different architecture. Unlike BERT or RoBERTa, T5 is an encoder–decoder model designed for text-to-text tasks. This allows the classification task to be restructured as a token prediction problem, where the model outputs label names as a string. The encoder still provides language understanding, while the decoder enables multi-label output as free-form text.

All three models were trained with a hyperparameter grid search to find the best performing model within the grid constraints.

4 BERT Baseline Model

4.1 Data Preprocessing

Label vectors were flattened into one-hot encoded lists aligned with the final label vocabulary. Text fields were cleaned by collapsing whitespace, converting to strings, stripping leading/trailing whitespace, and concatenating fields using `[SEP]` tokens:

```
{
  "text": "Title: ... [SEP] Body: ... [
      SEP] Comments: \[...\]",
  "labels": [1.0, 0.0, ..., 1.0]
}
```

The datasets were then tokenized using the `bert-base-uncased` tokenizer.

4.2 Baseline Results

The best-performing baseline BERT model was selected based on the micro F1 score on the validation set due to the class imbalance in the dataset. The model was fine-tuned for 3 epochs with a batch size of 16. The first 6 encoder layers were frozen. See Table 9 for results on the test set.

Overall the micro F1 (**0.8605**), precision (0.8771), and recall (0.8447) scores on the test set are fairly high. However, the macro F1 (0.6330), precision (0.7179), and recall (0.6238) scores are much lower, indicating a large number of false positives and false negatives for labels that have fewer examples in the dataset. The accuracy scores, which are high for both partial (0.9835) and complete (0.8168) correctness of multiple predicted labels on an issue, indicate good performance for the baseline model. Even so, subset accuracy is quite a bit lower than the partial accuracy, likely due to the class imbalance causing less-present labels to not be predicted correctly and reducing the number of issues for which all labels are correctly predicted.

To further understand how the class imbalance affects model performance, label-wise precision, recall, and F1 scores were calculated. The `bug` label, which is the most frequent in the dataset, showed strong performance with an F1 of 0.9062, while lower-frequency labels such as `security` and `ui` had no true positives, leading to 0 scores for precision, recall, and F1. A breakdown of the label scores is provided in Table 3.

Label	Precision	Recall	F1
bug	0.9057	0.9068	**0.9062**
featurerequest	0.8684	0.8681	0.8682
enhancement	0.7946	0.7621	0.7780
accessibility	0.9158	0.8894	0.9024
revert	0.9571	0.9936	0.9750
good_first_issue	0.7582	0.3520	0.4808
api	0.5943	0.5843	0.5892
help_wanted	0.6500	0.1566	0.2524
dependency	1.0000	1.0000	1.0000
webview	0.8750	0.8289	0.8514
feature	0.5392	0.5978	0.5670
oslinux	0.9111	0.9318	0.9213
regression	1.0000	0.0423	0.0811
documentation	0.7581	0.6812	0.7176
test	0.7273	0.8333	0.7767
release	0.6667	0.8000	0.7273
ui	0.0000	0.0000	0.0000
security	0.0000	0.0000	0.0000

Table 3: Per-label performance of the BERT model, sorted by label frequency in the dataset.

Looking at Table 4, it appears there may be some slight overfitting given how much better the metrics are when evaluating on the training set vs. the validation and test sets.

Metric	Train	Validation	Test
Micro Precision	0.9438	0.8834	0.8771
Micro Recall	0.9116	0.8487	0.8447
Micro F1-score	**0.9274**	**0.8657**	**0.8606**
Macro Precision	0.8408	0.7294	0.7179
Macro Recall	0.6830	0.6095	0.6238
Macro F1-score	0.7017	0.6352	0.6330
Mean Accuracy	0.9914	0.9841	0.9835
Subset Accuracy	0.8905	0.8175	0.8168
Loss	0.0272	0.0481	0.0485

Table 4: Performance of the baseline model across training, validation, and test sets.

5 RoBERTa

5.1 Data Preprocessing

Preprocessing was the same as for the baseline model, aside from using the `roberta-base` tokenizer (derived from the GPT-2 tokenizer).

5.2 RoBERTa Results

The micro F1 score on the validation set was again used to select the best RoBERTa model due to the class imbalance in the dataset. The best RoBERTa model was fine-tuned for 3 epochs with a batch size of 8. The first 6 encoder layers were frozen. The test set performance of the model is recorded in Table 9.

The results follow a similar pattern to the baseline model, surfacing the class imbalance in the dataset. However, unexpectedly, the performance is worse than the baseline model with a micro F1 score of **0.8230**. Table 5 indicates that the RoBERTa model is affected to a greater degree by the class imbalance, with more label classes displaying 0 true positives. This is reflected in the especially low macro F1 (0.4543), precision (0.5262), and recall (0.4334) metrics.

Label	Precision	Recall	F1
bug	0.8721	0.8936	**0.8827**
featurerequest	0.8515	0.8238	0.8374
enhancement	0.7666	0.6654	0.7124
accessibility	0.8841	0.8438	0.8635
revert	0.9207	0.9618	0.9408
good_first_issue	0.5200	0.0663	0.1176
api	0.6481	0.3933	0.4895
help_wanted	0.0000	0.0000	0.0000
dependency	0.9943	1.0000	0.9971
webview	0.8163	0.7895	0.8027
feature	0.5000	0.4239	0.4588
oslinux	0.8642	0.7955	0.8284
regression	0.0000	0.0000	0.0000
documentation	0.8333	0.1449	0.2469
test	0.0000	0.0000	0.0000
release	0.0000	0.0000	0.0000
ui	0.0000	0.0000	0.0000
security	0.0000	0.0000	0.0000

Table 5: Per-label performance of the RoBERTa model, sorted by label frequency in the dataset.

The worse performance for this model could indicate poorly chosen hyperparameters, a problem with the pre-processing, tokenization, or training code, or extra sensitivity to the dataset class imbalance. From Table 6, there doesn't appear to be much overfitting given the model's performance on the training set against the test and validation sets. This could indicate that the models need to be trained for more epochs for better performance.

6 T5

6.1 Data Preprocessing

Unlike the BERT-based models, T5 requires a single target text string for each issue rather than a one-hot list indicating whether each label applies. The datasets were reformatted such that the target text for each issue was a space-separated string containing each label applied to the issue:

Metric	Train	Validation	Test
Loss	0.0473	0.0610	0.0615
Micro Precision	0.9053	0.8654	0.8572
Micro Recall	0.8374	0.7959	0.7914
Micro F1-score	**0.8701**	**0.8292**	**0.8230**
Macro Precision	0.6076	0.5890	0.5262
Macro Recall	0.4633	0.4316	0.4334
Macro F1-score	0.4914	0.4555	0.4543
Mean Accuracy	0.9850	0.9802	0.9795
Subset Accuracy	0.8246	0.7819	0.7778

Table 6: Performance of the RoBERTa model across training, validation, and test sets.

```
{
  "input_text": "Title: ... [SEP] Body:
      ... [SEP] Comments: [...]",
  "target_text": "label_1 label_3 ..."
}
```

The labels were ordered in the same way as the one-hot list used with the BERT models. The datasets were tokenized using the `t5-base` tokenizer.

6.2 Adapting for Classification Metrics

Due to the text-to-text nature of the T5 model, additional processing was required to adapt it to this multi-class classification problem and calculate the same metrics. Post-processing steps for each generated output involved splitting the string on whitespace, then creating the one-hot list of values matching what was output by the BERT-based models depending on the labels present in the list of output words.

Once the output was converted to the one-hot list, the same metrics as the BERT-based models were calculated.

6.3 T5 Results

The best T5 model was also chosen based on the derived micro F1 score on the validation set. The best T5 model achieved a Micro F1-score of **0.8507** on the test set, which was competitive with the BERT baseline and better than the RoBERTa model. Similar issues to the BERT models are seen with the macro scores as well. However, even with the issues due to class imbalance, the T5 model generally performed better with less-present labels as displayed in the per-class performance in Table 7, with only `ui` having no true positive predictions.

In addition to classification metrics, the T5 model was evaluated using BLEU and ROUGE scores due to its generative output. The model achieved a BLEU score of 0.6602, indicating a somewhat high degree of n-gram overlap between generated label strings and the actual label strings. ROUGE scores were higher, with ROUGE-1 (0.8570) and ROUGE-L (0.8573) both above 0.85. This aligns with the mean (partial) accuracy for labels where single-label issues are correct, and the high ROUGE-L score is likely due to an abundance of single-label issues which count for a longest

Label	Precision	Recall	F1
bug	0.8937	0.9103	**0.9019**
featurerequest	0.8499	0.8676	0.8587
enhancement	0.7901	0.7416	0.7651
accessibility	0.8793	0.8582	0.8686
revert	0.9410	0.9650	0.9528
good_first_issue	0.6132	0.3316	0.4305
api	0.7130	0.4326	0.5385
help_wanted	0.7000	0.0422	0.0795
dependency	0.9886	1.0000	0.9943
webview	0.9071	0.8355	0.8699
feature	0.0210	0.5761	0.0405
oslinux	0.9481	0.8295	0.8848
regression	0.3636	0.0563	0.0976
documentation	0.8000	0.6957	0.7442
test	0.6406	0.8542	0.7321
release	0.8333	1.0000	0.9091
ui	0.0000	0.0000	0.0000
security	1.0000	0.1000	0.1818

Table 7: Per-label performance of the T5 model, sorted by label frequency in the dataset.

common subsequence in addition to some multi-label issues which are correct. The ROUGE-2 score, which emphasizes bigram overlap, was significantly lower at 0.0253, likely due to the sparsity of issues with two or more labels shown in Table 2.

Metric	Train	Validation	Test
Loss	0.0044	0.0052	0.0051
Micro Precision	0.8873	0.8726	0.8664
Micro Recall	0.8552	0.8366	0.8355
Micro F1-score	**0.8710**	**0.8542**	**0.8507**
Macro Precision	0.7597	0.7478	0.7413
Macro Recall	0.6207	0.5799	0.6086
Macro F1-score	0.6539	0.6183	0.6259
Mean Accuracy	0.9848	0.9828	0.9823
Subset Accuracy	0.8372	0.8154	0.8154
ROUGE-1	0.8760	0.8598	0.8570
ROUGE-2	0.0270	0.0241	0.0253
ROUGE-L	0.8761	0.8595	0.8573
BLEU	0.6761	0.6593	0.6602

Table 8: Performance of the T5 model across training, validation, and test sets.

Looking at Table 8, it appears that the T5 model displays fewer signs of overfitting than either Bert-based model, indicating that training for more epochs could yield greater performance gains while maintaining generalizability.

7 Discussion of Results

The BERT baseline model achieved the strongest overall classification performance, with a micro F1-score of **0.8606**, macro F1-score of **0.6330**, and subset accuracy of **0.8168** on the test set. These results suggest that BERT effectively captured patterns in common labels. However, over 90% of issues in the dataset contained only a single label. As a result, high subset accuracy is easier to achieve and may not reflect true multi-label prediction capability. In addition, its macro F1-score, while better than the other models, still reflects a drop in performance on underrepresented labels such as `security` and `ui`.

Metric	BERT	RoBERTa	T5
Micro Precision	**0.8771**	0.8572	0.8664
Micro Recall	**0.8447**	0.7914	0.8355
Micro F1-score	**0.8606**	0.8230	0.8507
Macro Precision	**0.7179**	0.5262	0.7413
Macro Recall	**0.6238**	0.4334	0.6086
Macro F1-score	**0.6330**	0.4543	0.6259
Mean Accuracy	**0.9835**	0.9795	0.9823
Subset Accuracy	**0.8168**	0.7778	0.8154
Test Loss	0.0485	0.0615	**0.0051**
ROUGE-1	–	–	0.8570
ROUGE-2	–	–	0.0253
ROUGE-L	–	–	0.8573
BLEU	–	–	0.6602

Table 9: Comparison of evaluation metrics across BERT, RoBERTa, and T5 models. Best values across models are bolded.

RoBERTa, though pre-trained on a larger corpus with improvements such as dynamic masking, underperformed relative to expectations. It achieved a micro F1-score of 0.8230 and a macro F1-score of 0.4543 on the test set, along with the highest test loss of 0.0615. It failed to make any correct predictions for several minority classes such as `release`, `security`, and `ui`. This suggests that RoBERTa struggled more with rare classes. The poor performance of this model may be partially due to suboptimal training conditions, including poorly-chosen hyperparameters. A wider grid search could potentially yield better results.

The T5 model delivered a test micro F1-score of 0.8507 and a macro F1-score of 0.6259, placing it between BERT and RoBERTa in classification performance. It achieved the lowest test loss (0.0051), and its predictions were also evaluated using generative metrics: a BLEU score of 0.6602, and ROUGE-1 and ROUGE-L scores of 0.8570 and 0.8573, respectively. These scores reflect strong overlap with the target label sequences. The low ROUGE-2 score (0.0253) is expected due to the sparsity of bigrams with the vast majority of issues having only a single label. While T5 performed well overall, its performance was similarly shaped by label imbalance and the dominance of single-label issues.

8 Conclusion

This work explored multi-label classification of GitHub issues using transformer-based models. By leveraging a carefully constructed dataset with rich textual input—including issue titles, descriptions, and comments, three model architectures (BERT, RoBERTa, and T5) were fine-tuned and evaluated. BERT demonstrated the strongest classification performance over-

all, while T5 performed competitively and enabled the use of generative metrics such as BLEU and ROUGE. RoBERTa underperformed, possibly due to suboptimal training or heightened sensitivity to class imbalance. While all models showed high precision on common labels, performance on rare labels remained limited, suggesting room for improvement in handling label imbalance. These findings highlight the importance of contextual understanding in issue classification.

8.1 Further Research Avenues

Several directions can be pursued to extend this work. Better techniques or further resampling could be used to significantly reduce the label imbalance in the dataset prior to training. In addition, the test dataset could be extended with labeled issues from currently unseen repositories by the models to further evaluate their generalizability.

Incorporating auxiliary data sources such as StackOverflow or related community forums could improve understanding of the language in the issues. Using the actual code from the repositories associated with the issues could provide useful context on what the issue text is referring to. Experimenting with BERT-specific input masking tokens like `[QUOTE]`, `[CODE]`, and `[URL]` may provide insight into how contextual cues influence classification performance.

A route to more dynamic labeling could start by studying how issue classification changes in real time as comments are added or removed.

Future work could also evaluate decoder-only models such as GPT-2 or LLaMA to explore generative performance on this task. In addition, other BERT variants such as DeBERTa or ELECTRA could be evaluated.

References

John Anvik, Lyndon Hiew, and Gail Murphy. 2006. Who should fix this bug? In *Proceedings - International Conference on Software Engineering 2006*, volume 2006, pages 361–370.

Deeksha Arya, Wenting Wang, Jin L. C. Guo, and Jinghui Cheng. 2019. Analysis and detection of information types of open source software issue discussions.

Shikhar Bharadwaj and Tushar Kadam. 2022. Github issue classification using bert-style models. In *2022 IEEE/ACM 1st International Workshop on Natural Language-Based Software Engineering (NLBSE)*, pages 40–43.

Anas Nadeem, Muhammad Usman Sarwar, and Muhammad Zubair Malik. 2021. Automatic issue classifier: A transfer learning framework for classifying issue reports. In *2021 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, page 421–426. IEEE.

Ye Paing, Tatiana Castro Vélez, and Raffi Khatchadourian. 2022. Quertci: A tool integrating github issue querying with comment classification.

Mohammed Latif Siddiq and Joanna C. S. Santos. 2022. Bert-based github issue report classification. In *2022 IEEE/ACM 1st International Workshop on Natural Language-Based Software Engineering (NLBSE)*, pages 33–36.

Tao Zhang and Byungjeong Lee. 2012. An automated bug triage approach: A concept profile and social network based developer recommendation. In *Intelligent Computing Technology*, pages 505–512, Berlin, Heidelberg. Springer Berlin Heidelberg.